

Lernziele:

- Aus Expression-Trees Assembler-Code erzeugen
- Wie in jedem Entwicklungszyklus: Code Refactoring
- Inkrementelle Softwareentwicklung und Vertiefung des bisher Gelernten

Schritt 1 (Parser: Code Review)

Seit der letzten Session unterstützt unser Parser folgende Grammatik:

```
expr = term { ( "+" | "-" ) term }
term = factor { ( "*" | "/" ) factor }
factor = decimal-literal | "(" expr ")"
```

Man kann damit beispielsweise den Ausdruck `2*(3+4)` parsen. Das Programm `xtest_parser` erzeugt dabei eine textuelle Darstellung des erzeugten Expression-Trees. Beispielsweise liefert `echo "2*(3+4)" | ./xtest_parser`

die Ausgabe

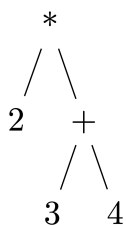
```
[ * [2] [ + [3] [4]] ]
```

Der Wurzelknoten des Baums enthält also den Multiplikationsoperator `*`. Der linke Teilbaum besteht aus dem Blattknoten `2`, der rechte Teilbaum beschreibt den Ausdruck `3+4`.

Zur besseren Veranschaulichung kann der Baum auch grafisch dargestellt werden. Mit

```
echo "2*(3+4)" | make tree.tex && lualatex tree.tex
```

wird eine LaTeX-Datei `tree.tex` und anschliessend `tree.pdf` erzeugt, die den Expression-Tree visualisiert. Öffnet man die PDF-Datei, erhält man beispielsweise die folgende Baumstruktur:



Bevor ihr mit den nächsten Schritten beginnt, solltet ihr euch den bisherigen Code von Parser und Expression-Tree noch einmal ansehen und sicherstellen, dass ihr versteht, wie die einzelnen Knoten erzeugt und miteinander verknüpft werden. Das wird für die Code-Generierung in den folgenden Schritten wichtig sein.

Bisher haben wir den Expression-Tree lediglich verwendet, um LaTeX-Code zur Visualisierung des Ausdrucks zu erzeugen. Im nächsten Schritt werden wir denselben Baum verwenden, um Assembler-Code zu generieren. Dazu werden wir den Baum erneut rekursiv durchlaufen und für jeden Knotentyp geeigneten Assembler-Code ausgeben.

Schritt 2 (Code aus Expression-Trees erzeugen)

In `expr.hdr` deklarieren wir analog zu `printExprTree` eine neue Funktion `loadExpr`:

```
// methods
extern fn printExprTree(expr: ->Expr);
extern fn loadExpr(expr: ->Expr): GenReg;
```

Die Funktion erwartet einen Zeiger auf einen Expression-Tree und liefert ein Register vom Typ `GenReg` zurück. Zweck der Funktion ist es, Assembler-Code zu erzeugen, der den Wert der Expression in ein freies Register lädt. Der Rückgabewert gibt an, welches Register den Wert der Expression nach Ausführung des erzeugten Assembler-Codes enthält.

Um die Funktion testen zu können, passen wir bereits jetzt `xtest_expr.abc` an. Nachdem der Expression-Tree erzeugt wurde und bevor er wieder freigegeben wird, fügen wir die folgende Zeile ein:

```
genReleaseReg(loadExpr(expr));
```

Die Funktion `loadExpr` reserviert während der Code-Erzeugung ein Register. Anschliessend wird mit `genReleaseReg` dieses Register wieder freigegeben.

Ein ausführbares Programm kann man zu diesem Zeitpunkt noch nicht erzeugen, da die Implementierung von `loadExpr` noch fehlt und der Linker daher mit einer Fehlermeldung abbrechen würde. Mit

```
abc -c xtest_expr.abc
```

kann man aber bereits versuchen, ein Objekt-File zu erzeugen. Zur Wiederholung: Die Option `-c` übersetzt das Programm nur bis zum Objekt-File und führt keinen Link-Schritt durch. Auf diese Weise könnt ihr bereits prüfen, ob sich die Änderungen fehlerfrei übersetzen lassen. Häufige Fehler sind Tippfehler, vergessene `include`-Direktiven oder eine fehlende Deklaration in einer Header-Datei. Insbesondere sollte `gen.hdr` sowohl in `expr.hdr` als auch in `xtest_expr.abc` eingebunden werden, da dort der Typ `GenReg` sowie die Funktion `genReleaseReg` verwendet werden. Erfahrene Softwareentwickler prüfen solche Abhängigkeiten meist selbstständig und verlassen sich nicht darauf, dass jede benötigte `include`-Direktive ausdrücklich erwähnt wird ;-)

Schritt 3 (Implementierung von `loadExpr`)

Für die Implementierung von `loadExpr` lohnt es sich, zunächst eine interne Hilfsfunktion zu schreiben. Diese soll Konstanten vom Typ `ExprKind` in Konstanten vom Typ `GenInstr` umwandeln.

Das ist ganz ähnlich wie im Parser: Dort wandelt die Funktion `exprKindFromToken` Konstanten vom Typ `TokenKind` in Konstanten vom Typ `ExprKind` um.

```
54 fn genInstrFromExprKind(exprKind: ExprKind): GenInstr
55 {
56     switch (exprKind) {
57         case EXPR_ADD: return GEN_ADD;
58         case EXPR_MINUS: return GEN_SUB;
59         case EXPR_MUL: return GEN_MUL;
60         case EXPR_DIV: return GEN_DIV;
61         default:
62             assert(0 && "internal error: genInstrFromInstrKind");
63             return -1;
64     }
65 }
```

Nun kann man sich an der Funktion `printExprTree` orientieren: Kopiert die Funktion, ändert den Namen und passt den Inhalt schrittweise an.

Bei einem Integer-Knoten soll nun nicht mehr der Wert ausgegeben werden. Stattdessen wird mit `genConstantAssembler`-Code erzeugt, der den Wert in ein freies Register lädt. Das Register, das `genConstant` zurückgibt, wird anschließend von `loadExpr` zurückgegeben.

Wie bei `printExprTree` wird der Expression-Tree rekursiv durchlaufen.

Bei einem Binary-Knoten wird zunächst für den linken und den rechten Teilbaum rekursiv `loadExpr` aufgerufen. Dadurch wird Code erzeugt, der die Werte der beiden Teilbäume jeweils in freie Register lädt. Danach können diese beiden Register wieder freigegeben werden, denn der Wert

des aktuellen Knotens wird anschließend in einem neuen freien Register berechnet.

Eine mögliche Implementierung ist:

```

67 fn loadExpr(expr: ->Expr): GenReg
68 {
69     if (expr->kind == EXPR_INTEGER) {
70         return genConstant(expr->decimal);
71     } else if (expr->kind >= EXPR_BINARY && expr->kind <= EXPR_BINARY_END) {
72         local left: GenReg = loadExpr(expr->left);
73         local right: GenReg = loadExpr(expr->right);
74         genReleaseReg(left);
75         genReleaseReg(right);
76
77         local op: GenInstr = genInstrFromExprKind(expr->kind);
78         return genInstr(op, right, left);
79     } else {
80         assert(0 && "loadExpr: not all cases handled!");
81         return genConstant("0");
82     }
83 }

```

Bei der Zeile

```
return genInstr(op, right, left);
```

fällt euch vielleicht die ungewöhnliche Reihenfolge der Argumente auf: `right` steht vor `left`. Das liegt an der Bedeutung (meiner Implementierung) von `genInstr`. Beispielsweise soll

```
genInstr(GEN_SUB, right, left)
```

Assembler-Code erzeugen, der den Wert im Register `right` vom Wert im Register `left` subtrahiert und das Ergebnis in einem neuen freien Register speichert. Es gilt also sinngemäß:

`left - right`

Aktuell verwenden wir allerdings noch die in `simple.isa` beschriebene Architektur. Diese kennt bisher nur die Instruktionen `halt`, `load`, `add` und `mul`. Für Addition und Multiplikation spielt die Reihenfolge der Operanden keine Rolle, sodass der Unterschied momentan noch nicht sichtbar wird.

Wenn wir die ISA später um weitere Instruktionen wie `sub` und `div` erweitern, müssen wir darauf achten, dass deren Semantik zu der Implementierung von `genInstr` passt. Deshalb ist es sinnvoll, solche Annahmen jetzt in einem Kommentar in `gen.hdr` zu dokumentieren.

Aufgabe: Implementiert `loadExpr` und testet die Funktion mit `xtest_expr`. Dokumentiert außerdem die Funktion `genInstr` in `gen.hdr` mit einem Kommentar.

Idealerweise würde man den Test direkt so erweitern, dass alle vier Operationen `+`, `-`, `*` und `/` getestet werden. Für den Moment sind wir hier aber etwas schlampig und verschieben diesen systematischen Test auf den nächsten Schritt.

Schritt 4 (Jetzt wird es langsam ein Compiler)

Passt `xtest_parser.abc` so an, dass nach dem Aufruf von `printExprTree` auch Assembler-Code für die Expression erzeugt wird.

Die Ausgabe von `printExprTree` können wir dabei in einen Kommentar schreiben:

```
printf("\t// expr: ");
printExprTree(expr);
printf("\n");
genReleaseReg(loadExpr(expr));
```

Mit

```
echo "(8-3)*4" | ./xtest_parser
```

erhält man dann eine Ausgabe der folgenden Form:

```
// expr: [ * [ - [8] [3]] [4]]
load 8,    %1
load 3,    %2
sub  %2,   %1,   %1
load 4,    %2
mul  %2,   %1,   %1
```

Jetzt sieht man zum ersten Mal deutlich, dass aus dem geparsen Ausdruck tatsächlich Assembler-Code erzeugt wird. Der Expression-Tree wird also nicht mehr nur angezeigt oder visualisiert, sondern dient als Zwischendarstellung für die Code-Erzeugung.

Schritt 5 (ISA erweitern)

Damit wir den erzeugten Assembler-Code später in Maschinencode übersetzen und anschließend ausführen können, müssen wir die Architekturbeschreibung in `simple.isa` erweitern.

Bei dieser Gelegenheit erweitern wir den Befehlssatz gleich um einige weitere Instruktionen, die wir später im Zusammenhang mit Funktionen benötigen werden. Außerdem weichen wir dabei bewusst etwas von dem Assembler-Dialekt ab, der in `ulm-ice40.isa` beschrieben wird. Beispielsweise verwenden wir Instruktionsnamen wie `add`, `sub`, `mul` und `div` statt `addq`, `subq`, `imulq` und `idivq`.

Auch die Speicherzugriffe werden etwas anders notiert. Die Instruktionen `fetch` und `store` besitzen einen Parameter `size`, der die Größe des Speicherzugriffs festlegt. Genauer gesagt gibt `size` den Exponenten an. Es werden also jeweils 2^{size} Bytes gelesen oder geschrieben.

Beispiele:

```
// lade  $2^3 = 8$  Bytes von Adresse %2 nach %3
fetch (%2, 3), %3
```

```
// lade  $2^3 = 8$  Bytes von Adresse %2 + 16 nach %3
fetch 16(%2, 3), %3
```

```
// speichere  $2^3 = 8$  Bytes aus %2 an Adresse %3
store %2, (%3, 3)
```

```
// speichere  $2^3 = 8$  Bytes aus %2 an Adresse %3 + 16
store %2, 16(%3, 3)
```

Der Wert `3` steht hier also für einen 8-Byte-Zugriff. Entsprechend würden die Werte `0`, `1` und `2` Zugriffe auf `1`, `2` bzw. `4` Bytes beschreiben.

Fügt dazu am Ende der Datei `simple.isa` die folgenden Zeilen hinzu (alternativ könnt ihr die aktuelle Version der Datei von Moodle übernehmen):

```
##
## new in session 17
##

U20_R      (OP u 8) (dest u 4) (imm s 20)
R           (OP u 8) (x u 4)
MR_R       (OP u 8) (data u 4) (addr u 4) (size u 2) (offset s 14)
R_MR       (OP u 8) (data u 4) (addr u 4) (size u 2) (offset s 14)
U16_R_R    (OP u 8) (z u 4) (y u 4) (imm u 16)

0x20 R_R_R
:   sub %x, %y, %z
    ulm_sub64(ulm_regVal(x), ulm_regVal(y), z);

0x21 R_R_R
:   div %x, %y, %z
    ulm_idiv64(ulm_regVal(x), ulm_regVal(y), z);

0x22 U16_R_R
:   add imm, %y, %z
    ulm_add64(imm, ulm_regVal(y), z);

0x23 U16_R_R
:   sub imm, %y, %z
    ulm_sub64(imm, ulm_regVal(y), z);

0x24   U20_R
:   call   imm, %dest
    ulm_absJump(imm, dest);

0x25 R
:   ret    %x
    ulm_absJump(ulm_regVal(x), x);

0x26 MR_R
:   fetch  offset(%addr, size), %data
:   fetch  (%addr, size), %data
    ulm_fetch64(offset, addr, 0, 1, ULM_ZERO_EXT, 1 << size, data);

0x27 R_MR
:   store  %data, offset(%addr, size)
:   store  %data, (%addr, size)
    ulm_store64(offset, addr, 0, 0, 1 << size, data);
```

Anschließend könnt ihr mit `make` die Hardware-Simulation und den Assembler neu erzeugen. Mit `ulm-generator --refman simple.isa`

wird außerdem die Datei `simple/refman.pdf` erzeugt. Diese enthält eine Referenzdokumentation aller Instruktionen der Architektur. Schaut euch die neuen Befehle ruhig kurz an. Ihr müsst noch nicht jedes Detail verstehen. Wichtig ist zunächst, dass die Architektur jetzt auch Instruktionen für Subtraktion, Division, Funktionsaufrufe und Speicherzugriffe unterstützt.

Schritt 6 (Code-Generator erweitern)

Aktuell kann unser Compiler bereits Assembler-Code aus einem arithmetischen Ausdruck erzeugen. Dieser Assembler-Code lässt sich auch in Maschinencode übersetzen und auf unserer einfachen Hardware-Simulation ausführen. Ein vollständiges Programm ist das aber noch nicht. Es fehlt beispielsweise noch eine halt-Instruktion, damit das Programm sauber terminiert. Wenn der Wert der Expression in Register %1 steht, könnte man zum Beispiel mit `halt %1` das Programm beenden und den Wert der Expression als Exit-Code zurückgeben.

Man könnte diese Instruktion nun einfach mit einer `printf`-Anweisung in `xtest_parser.abc` ausgeben. Das wäre aber nur eine schnelle Notlösung. Stattdessen erweitern wir unseren Code-Generator um Funktionen, die wir ohnehin bald benötigen werden.

Wie ihr bereits in Blatt 14 gesehen habt, wird einiges an *Boilerplate-Code* benötigt, etwa zum Initialisieren des Stacks sowie für Function-Prologue und Function-Epilogue.

In `gen.hdr` kommen in diesem Schritt die folgenden Funktionsdeklarationen dazu:

```
extern fn genInitCode();
extern fn genFuncDefBegin(name: -> readonly char);
extern fn genFuncDefEnd();
extern fn genHalt(exitCode: GenReg);
extern fn genReturn(retVal: GenReg);
```

Den Code für `genHalt`, `genInitCode`, `genFuncDefBegin` und `genFuncDefEnd` könnt ihr bei Moodle herunterladen oder weiter unten abtippen. Die einzige Herausforderung bei diesen Funktionen besteht darin, die notwendige Geduld aufzubringen und daran zu denken, wie man ein Prozentzeichen in einem String-Literal ausgibt: In einem Format-String von `printf` schreibt man dafür `%%`.

Die Implementierung von `genInitCode` sieht so aus:

```
fn genInitCode()
{
    printf("\t.text\n");
    printf("\t.equ \tSP, \t1\n");
    printf("\tload \t0, \t%%SP\n");
    printf("\tsub \t1*8, \t%%SP, \t%%SP\n");
    printf("\tcall \tmain, \t%%2\n");
    printf("\tfetch \t(%%SP, 3), \t%%2\n");
    genHalt(2);
}
```

Wenn eure Implementierung von `genHalt` richtig ist, erzeugt ein Aufruf von `genInitCode` die folgende Ausgabe:

```
.text
.equ    SP,      1
load    0,       %SP
sub      1*8,     %SP,    %SP
call     main,    %2
fetch    (%SP, 3), %2
halt     %2
```

Die Implementierung von `genFuncDefBegin` sieht so aus:

```
fn genFuncDefBegin(name: -> char)
{
    printf("// function %s\n", name);
    printf("%s:", name);
    printf("\tsub \t8, \t%%SP, \t%%SP\n");
    printf("\tstore \t%%2, \t(%%SP, 3)\n");
    printf("\t// begin of function body\n");
}
```

Ein Aufruf `genFuncDefBegin("main")` erzeugt beispielsweise:

```
// function main
main:  sub      8,      %SP,    %SP
       store    %2,     (%SP, 3)
       // begin of function body
```

Die Implementierung von `genFuncDefEnd` sieht so aus:

```
fn genFuncDefEnd()
{
    printf("\t// end of function body\n");
    printf("\tfetch \t(%SP, 3), \t%2\n");
    printf("\tadd \t8, \t%SP, \t%SP\n");
    printf("\tret \t%2\n");
}
```

Ein Aufruf erzeugt:

```
// end of function body
fetch  (%SP, 3),  %2
add    8,  %SP,  %SP
ret    %2
```

Aufgabe

1. Passt `gen.hdr` und `gen.abc` wie oben beschrieben an. Bevor ihr die in `gen.abc` noch fehlenden Implementierungen ergänzt, sollt ihr bereits `xtest_parser.abc` anpassen. Dadurch habt ihr von Anfang an einen einfachen Test für die neuen Funktionen. Die ersten beiden Anweisungen in `main` sollen sein:

```
genInitCode();
genFuncDefBegin("main");
```

Mit `genInitCode` wird Code erzeugt, der den Stackpointer initialisiert, die Funktion `main` anspringt und deren Rückgabewert später als Exit-Code verwendet.

Mit `genFuncDefBegin("main")` wird Boilerplate-Code für den Function-Prologue von `main` ausgegeben.

Danach erzeugt euer Compiler wie bisher Code für eine Expression, die der Benutzer eingeben kann. Die letzten beiden Anweisungen in `main` sollen dann sein:

```
genReturn(2);
genFuncDefEnd();
```

Mit `genReturn(2)` wird der Inhalt von `%2` als Rückgabewert auf den Stack gelegt.

Mit `genFuncDefEnd` wird der Function-Epilogue erzeugt.

2. Implementiert `genHalt`, sodass eine `halt`-Instruktion ausgegeben wird. Das durch `exitCode` spezifizierte Register soll dabei als Exit-Code verwendet werden. Hat `exitCode` beispielsweise den Wert `2`, soll also

```
halt %2
```

ausgegeben werden.

3. Implementiert `genReturn`, sodass das durch `retVal` spezifizierte Register als Rückgabewert auf den Stack gelegt wird. Der Rückgabewert soll dabei an Stack-Offset 8 gespeichert werden. Hat `retVal` beispielsweise den Wert `2`, soll also

```
store %2, 8(%SP, 3)
```

ausgegeben werden.

4. Der von `xtest_parser` erzeugte Code sollte sich jetzt assemblieren lassen. Testet das mit

```
echo "6*(3+4)" | ./xtest_parser > foo.s
./simple/ulmas foo.s && ./simple/ulm a.out; echo $?
```

Allerdings wird es beim Ausführen des Programms noch zu einem Crash kommen

5. Für den Stackpointer verwenden wir das Register `%1`. Das wurde im Code, den `genInitCode` erzeugt, so festgelegt. Dann darf `%1` aber nicht mehr für die Berechnung einer Expression verwendet werden. Passt daher in `gen.abc` die Funktion `getReg` so an, dass nur Register größer als `1` zurückgeliefert werden. Danach sollte euer Compiler korrekten Code erzeugen. Wiederholt den Test aus der vorigen Teilaufgabe. Der Exit-Code sollte nun `42` sein.
6. Bei dieser Gelegenheit können wir noch das Interface verbessern. Ändert in `gen.hdr` die Deklaration von `genConstant` zu

```
extern fn genConstant(val: -> readonly char): GenReg;
```

Passt anschließend in `gen.abc` die Implementierung entsprechend an.

Schritt 7 (Vorbereitung für den nächsten großen Schritt)

Aktuell ist unser Compiler noch ein ziemlich schlechter Taschenrechner: Er kann arithmetische Ausdrücke aus Konstanten parsen und dafür Maschinencode erzeugen. Eine der wichtigsten Zutaten, die noch fehlen, sind *Variablen* und *Zuweisungen*. Das werden wir in Session 19 angehen. Die in Session 17 entwickelte Datenstruktur für Symboltabellen können wir aber schon einmal übernehmen und für unsere Zwecke anpassen. Kopiert dazu aus Session 17 die Dateien `syntab.hdr` und `syntab.abc` in euer aktuelles Projektverzeichnis.

Ergänzt anschließend `syntab.hdr` um die folgende Struktur und die beiden Funktionsdeklarationen:

```
struct SyntabInfo
{
    isGlobal: bool;
    offset: int;
};

extern fn syntabGetInfo(ident: String, info: -> SyntabInfo): bool;
extern fn syntabIsDefined(ident: String): bool;
```

Da ihr die Symboltabelle erst in der letzten Session implementiert habt, solltet ihr die beiden neuen Funktionen mit wenig zusätzlicher Hilfe ergänzen können. Der folgende Test zeigt, welches Verhalten erwartet wird. Legt dazu ein Testprogramm `xtest_syntab.abc` mit folgendem Inhalt an (Sourcecode gibt's auf Moodle):

```
@ <stdio.hdr>

@ "string.hdr"
@ "syntab.hdr"

fn checkInfo(ident: String) {
    local info: SyntabInfo;
    printf("%s is %s defined\n", ident, syntabIsDefined(ident) ? "" : "not");
    if (syntabGetInfo(ident, &info)) {
        printf("%s is %s\n", ident, info.isGlobal then "global" else "local");
        if (!info.isGlobal) {
            printf("... with offset: %d\n", info.offset);
        }
    } else {
        printf("unknown identifier %s\n", ident);
    }
}

fn main()
{
    // add some global and local variables
    syntabAddGlobal((String)"foo") || printf("error\n");
    syntabAddLocal((String)"bar", -8) || printf("error\n");

    checkInfo((String)"foo");
    checkInfo((String)"bar");
    checkInfo((String)"barx");
}
```

Das Programm soll die folgende Ausgabe erzeugen:

```
foo is defined
foo is global
bar is defined
bar is local
... with offset: -8
barx is not defined
unknown identifier barx
```

Aufgabe

Implementiert `syntabGetInfo` und `syntabIsDefined` so, dass dieser Test erfolgreich durchläuft.